动态规划竞赛板子

本板子默认 C++17。代码优先保证正确性、复杂度和赛场可抄性，变量名尽量短；所有 64 位整数统一使用 `ll`。

0. 公共约定与选型

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

const ll INF = (1LL << 62);
const int MOD = 1000000007;
```

`INF` 留有加法余量，但做最短路式转移时仍要先判断来源状态可达。若答案或中间乘积可能超过 `ll`，用 `__int128` 暂存。

问题	常用算法	时间复杂度
01 / 完全背包	一维 DP	`O(nW)`
多重背包	单调队列优化	`O(nW)`
可达重量集合	`bitset` 01 背包	`O(nW/64)`
LIS	二分维护结尾	`O(n log n)`
LCS / 编辑距离	双序列 DP	`O(nm)`
石子合并、回文等	区间 DP	常见 `O(n^3)`
数位限制计数	数位 DP	`O(位数 × 状态数 × 进制)`
小规模集合问题	状压 DP	常见 `O(n2^n)` 或 `O(3^n)`
少量终端的最小连通子图	Steiner Tree DP	`O(3^k n+2^k(m+n)log n)`
所有子集/超集聚合	SOS DP	`O(n2^n)`
树上选择、计数	树形 DP / 树上背包	`O(n)` 至 `O(nW^2)`
DAG 最优路径	拓扑 DP	`O(n+m)`
形如区间最优决策	分治 / Knuth 优化	视条件降至 `O(kn log n)` / `O(n^2)`
线性函数取最值	单调凸包 / Li Chao	`O(n)` / `O(n log C)`

0.1 建模检查表

1. 明确状态恰好表示什么；“不超过”与“恰好”不能混用。
2. 写出最后一个决策，确认所有来源状态已被计算。
3. 最大值不可达初始化为 `-INF`，最小值不可达初始化为 `INF`，计数初始化为 `0`。
4. 压缩维度前确认遍历方向：01 背包倒序，完全背包正序。
5. 原地更新时确认本轮是否允许再次使用刚更新的状态。
6. 计数题先判断空方案是否合法；取模乘法使用 `ll`。

0.2 常用初始化

```
// 最大值且要求恰好用 j 容量
vector<ll> f(W + 1, -INF);
f[0] = 0;

// 最大值且只要求容量不超过 W
vector<ll> f(W + 1, 0);

// 最小值且要求恰好达到 j
vector<ll> f(W + 1, INF);
f[0] = 0;

// 方案数：空集达到 0，故 f[0]=1
vector<ll> f(W + 1);
f[0] = 1;
```

---

1. 背包 DP

本节默认所有物品体积为正整数。体积为 `0` 的物品需单独处理，尤其完全背包中 `w=0,v>0` 会使答案无界。

1.1 01 背包

每件物品最多选一次。`w[i]` 为体积，`v[i]` 为价值，求总体积不超过 `W` 的最大价值。

- 状态：`f[j]` 表示容量不超过 `j` 时的最大价值。
- 参数：`w,v` 均从 `0` 开始且等长，`W` 为容量。
- 返回：最大价值 `f[W]`。
- 复杂度：时间 `O(nW)`，空间 `O(W)`。

倒序枚举容量保证一件物品不会在同一轮重复使用。

```cpp
ll knap01(const vector<int>& w, const vector<ll>& v, int W) { // w: 体积; v: 价值; W: 容量
    vector<ll> f(W + 1);
    for (int i = 0; i < (int)w.size(); i++)
        for (int j = W; j >= w[i]; j--)
            f[j] = max(f[j], f[j - w[i]] + v[i]);
    return f[W];
}
```

若要求总体积**恰好**为 `W`，必须把除 `f[0]` 外的状态初始化为 `-INF`：

```cpp
vector<ll> f(W + 1, -INF);
f[0] = 0;
for (int i = 0; i < n; i++)
    for (int j = W; j >= w[i]; j--)
        if (f[j - w[i]] != -INF)
            f[j] = max(f[j], f[j - w[i]] + v[i]);
// f[W]==-INF 表示无法恰好装满
```

1.2 完全背包

每种物品可选无限次。与 01 背包唯一的核心差别是容量正序，使本轮刚更新的状态可以继续转移。

- 参数和返回值同 `knap01`。
- 要求所有 `w[i]>0`；若存在 `w=0,v>0`，最大值无界。
- 复杂度：时间 `O(nW)`，空间 `O(W)`。

```cpp
ll knap_unbounded(const vector<int>& w, const vector<ll>& v, int W) { // 每种可无限选
    vector<ll> f(W + 1);
    for (int i = 0; i < (int)w.size(); i++)
        for (int j = w[i]; j <= W; j++)
            f[j] = max(f[j], f[j - w[i]] + v[i]);
    return f[W];
}
```

1.3 多重背包：二进制拆分

第 `i` 种物品最多选 `c[i]` 件。把数量拆成 `1,2,4,...` 个一组，每组视为一件 01 物品。代码短，适合中等规模。

- `w[i],v[i],c[i]`：体积、单件价值、数量。
- 复杂度：`O(W sum(log(c[i]+1)))`，空间 `O(W)`。

```cpp
ll knap_multi_bin(const vector<int>& w, const vector<ll>& v,
                  const vector<int>& c, int W) { // c[i]: 最多件数
    vector<ll> f(W + 1);
    for (int i = 0; i < (int)w.size(); i++) {
        if (w[i] > W) continue;
```

```
        int s = min(c[i], W / w[i]);            // 超过容量可容纳数的部分无用
        for (ll k = 1; s > 0; k <<= 1) {
            int t = min<ll>(k, s), x = w[i] * t;
            ll y = v[i] * t;
            for (int j = W; j >= x; j--)
                f[j] = max(f[j], f[j - x] + y);
            s -= t;
        }
    }
    return f[W];
}
```

1.4 多重背包：单调队列 `O(nW)`

按容量对 `w` 取模分组。对同一余数写成 `j=r+kw`，转移为

$$f'[r+kw] = kv + \max_{k-c\le t\le k}(f[r+tw] - tv),$$

窗口最大值可用单调队列维护。相比二进制拆分，在数量很大时更稳定。

- `add(w,v,c)`：向当前 DP 加入一种体积 `w`、价值 `v`、最多 `c` 件的物品。
- `f[j]` 始终表示容量不超过 `j` 的最大价值。
- 单种物品 `O(W)`，总时间 `O(nW)`，空间 `O(W)`。

```
struct MultiKnap {
    int W;
    vector<ll> f, g;
    vector<int> q;
    MultiKnap(int W) : W(W), f(W + 1), g(W + 1), q(W + 1) {}

    void add(int w, ll v, int c) { // w: 体积; v: 单价; c: 数量
        g = f;
        for (int r = 0; r < w && r <= W; r++) {
            int h = 0, t = -1, k = 0;
            for (int j = r; j <= W; j += w, k++) {
                while (h <= t && q[h] < k - c) h++;
                ll x = g[j] - (ll)k * v;
                while (h <= t) {
                    int p = q[t], jp = r + p * w;
                    if (g[jp] - (ll)p * v > x) break;
                    t--;
                }
                q[++t] = k;
                int p = q[h];
                f[j] = g[r + p * w] + (ll)(k - p) * v;
            }
        }
    }
};
```

此模板允许“不装满”，因此初值全为 `0`。若要求恰好装满，将 `f[1..W]` 设为 `-INF`，并在计算队列键值时跳过不可达状态，不能直接对 `-INF` 做减法。

1.5 混合背包

同一题同时包含 01、完全和多重物品时分别转移。约定 `c=-1` 为 01，`c=0` 为无限，其余正数为件数。

```
vector<ll> f(W + 1);
for (int i = 0; i < n; i++) { // w,v,c: 当前物品
    cin >> w >> v >> c;
    if (w > W) continue;
    if (c == -1) {
        for (int j = W; j >= w; j--) f[j] = max(f[j], f[j - w] + v);
    } else if (c == 0) {
        for (int j = w; j <= W; j++) f[j] = max(f[j], f[j - w] + v);
    } else {
        c = min(c, W / w);
        for (ll k = 1; c > 0; k <<= 1) {
            int t = min<ll>(k, c), x = w * t;
            ll y = v * t;
            for (int j = W; j >= x; j--) f[j] = max(f[j], f[j - x] + y);
            c -= t;
        }
    }
}
```

1.6 分组背包

物品分组，每组最多选一件。必须先枚举容量倒序，再枚举组内物品，保证不会从同组的新状态继续转移。

- `g[i]` 存第 `i` 组的 `{体积,价值}`。
- 返回容量不超过 `W` 的最大价值。
- 复杂度 `O(W × 物品总数)`，空间 `O(W)`。

```
ll group_knap(const vector<vector<pair<int, ll>>>& g, int W) { // g[i]: 第 i 组
    vector<ll> f(W + 1);
    for (auto& a : g)
        for (int j = W; j >= 0; j--)
            for (auto [w, v] : a)
                if (j >= w) f[j] = max(f[j], f[j - w] + v);
    return f[W];
}
```

若每组必须恰选一件，使用上一层副本 `g=f`，把新层初始化为 `-INF`，只从 `g[j-w]` 转移。

1.7 二维费用 01 背包

每件物品消耗两种资源 `a[i],b[i]`，上限分别为 `A,B`。两维都倒序。

- 状态：`f[x][y]` 为消耗不超过 `x,y` 的最大价值。
- 复杂度：时间 `O(nAB)`，空间 `O(AB)`。

```
vector<vector<ll>> f(A + 1, vector<ll>(B + 1));
for (int i = 0; i < n; i++)                  // a[i],b[i],v[i]: 当前物品
    for (int x = A; x >= a[i]; x--)
        for (int y = B; y >= b[i]; y--)
            f[x][y] = max(f[x][y], f[x - a[i]][y - b[i]] + v[i]);
```

1.8 01 背包方案数

求恰好装到各容量的方案数，不关心价值。`f[0]=1` 表示空集；倒序保证每件只使用一次。

```
vector<ll> f(W + 1);
f[0] = 1;
for (int w : a)                              // a: 每件物品的体积
    for (int j = W; j >= w; j--)
        f[j] = (f[j] + f[j - w]) % MOD;
// f[W]: 恰好达到 W 的子集数
```

完全背包方案数只需把 `j` 改为从 `w` 到 `W` 正序。物品即使体积相同也按不同物品区分；若题目按数值多重集合去重，需要另外建模。

1.9 最大价值及最优方案数

同时维护最大值 `f[j]` 和达到该最大值的方案数 `c[j]`。下面求**恰好**装到每个容量的最优方案数。

```
vector<ll> f(W + 1, -INF), c(W + 1);
f[0] = 0, c[0] = 1;
for (int i = 0; i < n; i++) {
    for (int j = W; j >= w[i]; j--) {
        if (f[j - w[i]] == -INF) continue;
        ll x = f[j - w[i]] + v[i];
        if (x > f[j]) f[j] = x, c[j] = c[j - w[i]];
        else if (x == f[j]) c[j] = (c[j] + c[j - w[i]]) % MOD;
    }
}
// f[W]: 最大价值; c[W]: 最优方案数
```

若目标是容量不超过 `W` 的全局最优值，应取 `mx=max(f[j])`，并累加所有满足 `f[j]==mx` 的 `c[j]`，而不是把所有 `c[j]` 初始化成 `1`。

1.10 01 背包还原字典序最小方案

物品编号为 `1..n`，要求输出编号序列字典序最小。反向计算 `f[i][j]`：只考虑编号 `i..n`、容量不超过 `j` 的最大价值。还原时若选 `i` 仍最优就优先选择它。

- 空间 `O(nW)`，时间 `O(nW)`。
- 若允许价值为负且必须选物品，需要按题意修改初值。

```
vector<vector<ll>> f(n + 2, vector<ll>(W + 1));
for (int i = n; i >= 1; i--)
    for (int j = 0; j <= W; j++) {
        f[i][j] = f[i + 1][j];
        if (j >= w[i]) f[i][j] = max(f[i][j], f[i + 1][j - w[i]] + v[i]);
    }

vector<int> ans;
for (int i = 1, j = W; i <= n; i++)
    if (j >= w[i] && f[i][j] == f[i + 1][j - w[i]] + v[i])
        ans.push_back(i), j -= w[i];
```

1.11 `bitset` 加速可达性背包

只问哪些总重量可达时，用位运算并行更新。每件物品一次：`f |= f << w`。

- `M` 必须是编译期常量且大于最大容量。
- 复杂度约 `O(nW/64)`，内存 `O(W)`，常数很小。

```
const int M = 2000005;
bitset<M> f;
f[0] = 1;
for (int w : a) f |= f << w;                    // a: 01 物品体积
// f[x]==1 表示总重量 x 可达
```

动态容量可用 `vector<ull>` 手写位移，但代码更长；固定上限时优先使用 `bitset`。

---

2. 序列与字符串 DP

2.1 LIS：最长严格上升子序列

`d[len-1]` 维护长度为 `len` 的严格上升子序列的最小末尾。新数替换第一个 `>=x` 的位置。

- `lis_len(a)`：返回最长严格上升子序列长度。
- 严格上升用 `lower_bound`；非严格上升改为 `upper_bound`。
- 复杂度 `O(n log n)`，空间 `O(n)`。

```cpp
int lis_len(const vector<ll>& a) { // a: 序列
    vector<ll> d;
    for (ll x : a) {
        auto it = lower_bound(d.begin(), d.end(), x);
        if (it == d.end()) d.push_back(x);
        else *it = x;
    }
    return d.size();
}
```

需要还原一条 LIS 时，`id[len]` 记该长度当前最优末尾的下标，`pre[i]` 记前驱：

```cpp
vector<int> lis_path(const vector<ll>& a) { // 返回一条严格 LIS 的下标，0-index
    int n = a.size();
    vector<ll> d;
    vector<int> id, pre(n, -1);
    for (int i = 0; i < n; i++) {
        int p = lower_bound(d.begin(), d.end(), a[i]) - d.begin();
        if (p == (int)d.size()) d.push_back(a[i]), id.push_back(i);
        else d[p] = a[i], id[p] = i;
        if (p) pre[i] = id[p - 1];
    }
    vector<int> ans;
    for (int x = id.back(); x != -1; x = pre[x]) ans.push_back(x);
    reverse(ans.begin(), ans.end());
    return ans;
}
```

空序列时 `id.back()` 非法；若输入可能为空，在调用前特判。模板返回的是某一条 LIS，不保证下标或值的字典序最小。

2.2 二维偏序最长链

对二元组 `(x,y)` 求严格递增链。先按 `x` 升序、`y` 降序排序，再对 `y` 求严格 LIS。`x` 相同的点因 `y` 降序，不会被错误地同时选入。

```cpp
sort(a.begin(), a.end(), [](auto A, auto B) { // a: pair<x,y>
    if (A.first != B.first) return A.first < B.first;
    return A.second > B.second;
});
vector<ll> d;
for (auto [x, y] : a) {
    auto it = lower_bound(d.begin(), d.end(), y);
    if (it == d.end()) d.push_back(y);
    else *it = y;
}
// d.size(): x,y 均严格递增的最长链
```

若偏序定义含等号，排序和二分规则必须随题意调整，不能机械套用。

2.3 LCS：最长公共子序列

`f[j]` 表示当前处理完 `a` 的前缀后，与 `b[0..j)` 的 LCS 长度。`pre` 保存更新前的左上角值。

- `lcs_len(a,b)`：字符串也可改为任意支持 `==` 的数组。
- 复杂度 `O(nm)`，空间 `O(m)`；让较短串作为 `b` 可省内存。

```
int lcs_len(const string& a, const string& b) { // a,b: 两个串
    vector<int> f(b.size() + 1);
    for (char x : a) {
        int pre = 0;
        for (int j = 1; j <= (int)b.size(); j++) {
            int old = f[j];
            if (x == b[j - 1]) f[j] = pre + 1;
            else f[j] = max(f[j], f[j - 1]);
            pre = old;
        }
    }
    return f[b.size()];
}
```

需要还原一条 LCS 时保留二维数组，从 `(n,m)` 逆推。相等时走左或上会影响得到哪一条，不自动保证字典序最小。

```
vector<vector<int>> f(n + 1, vector<int>(m + 1));
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        f[i][j] = (a[i - 1] == b[j - 1]) ? f[i - 1][j - 1] + 1
                                         : max(f[i - 1][j], f[i][j - 1]);
string s;
for (int i = n, j = m; i && j; ) {
    if (a[i - 1] == b[j - 1]) s += a[i - 1], i--, j--;
    else if (f[i - 1][j] >= f[i][j - 1]) i--;
    else j--;
}
reverse(s.begin(), s.end());
```

2.4 LCS 的位并行优化

当字符集不大且只求 LCS 长度，可用位并行算法。位集 `s` 的第 `j` 位表示 DP 相邻位置的差分。

- 复杂度约 `O(nm/64)`；模板适合 `m` 很大、普通 `O(nm)` 过不了的场景。
- 使用动态 64 位块，无需预先写死字符串长度上限。

```
int lcs_bit64(const string& a, const string& b) { // 返回 LCS 长度
    int m = b.size(), K = (m + 63) >> 6;
    vector<vector<ull>> p(256, vector<ull>(K));
    for (int j = 0; j < m; j++) p[(unsigned char)b[j]][j >> 6] |= 1ULL << (j & 63);
    vector<ull> s(K), x(K), y(K), z(K);
```

```
    for (unsigned char c : a) {
        ull car = 1, bor = 0;
        for (int k = 0; k < K; k++) {
            x[k] = s[k] | p[c][k];
            ull ns = s[k] << 1 | car;
            car = s[k] >> 63, y[k] = ns;
            ull t = x[k] - y[k] - bor;
            ull nb = bor ? x[k] <= y[k] : x[k] < y[k];
            z[k] = t, bor = nb;
        }
        for (int k = 0; k < K; k++) s[k] = x[k] & ~z[k];
    }
    int ans = 0;
    for (auto x : s) ans += __builtin_popcountll(x);
    return ans;
}
```

2.5 编辑距离

允许插入、删除、替换一个字符，每次代价为 `1`，求把 `a` 变成 `b` 的最少操作数。

- `f[j]` 为当前行 `a` 的前缀变到 `b[0..j)` 的最小代价。
- 复杂度 `O(nm)`，空间 `O(m)`。

```
int edit_dist(const string& a, const string& b) { // a -> b
    int m = b.size();
    vector<int> f(m + 1);
    iota(f.begin(), f.end(), 0);
    for (int i = 1; i <= (int)a.size(); i++) {
        int pre = f[0]; f[0] = i;
        for (int j = 1; j <= m; j++) {
            int old = f[j];
            f[j] = min({f[j] + 1, f[j - 1] + 1,
                        pre + (a[i - 1] != b[j - 1])});
            pre = old;
        }
    }
    return f[m];
}
```

若三种操作代价不同，把三个 `+1` 替换为对应代价。若只关心距离是否 `<=k`，可只计算主对角线宽度 `2k+1` 的带状区域，将时间降为 `O(k min(n,m))`。

2.6 LCIS：最长公共上升子序列

同时是 `a,b` 的子序列且严格上升。`f[j]` 表示以 `b[j]` 结尾的 LCIS 长度；扫描一行时维护满足 `b[j]<a[i]` 的最大旧状态。

- 复杂度 `O(nm)`，空间 `O(m)`。

```cpp
int lcis(const vector<int>& a, const vector<int>& b) { // 严格上升
    vector<int> f(b.size());
    for (int x : a) {
        int mx = 0;
        for (int j = 0; j < (int)b.size(); j++) {
            if (b[j] < x) mx = max(mx, f[j]);
            else if (b[j] == x) f[j] = max(f[j], mx + 1);
        }
    }
    return b.empty() ? 0 : *max_element(f.begin(), f.end());
}
```

2.7 最大子段和

Kadane DP。`f` 表示必须以当前位置结尾的最大和；适用于全负数组。

```cpp
ll max_subarray(const vector<ll>& a) { // a 非空
    ll f = a[0], ans = a[0];
    for (int i = 1; i < (int)a.size(); i++)
        f = max(a[i], f + a[i]), ans = max(ans, f);
    return ans;
}
```

需要区间端点时，在选择 `a[i]` 重新开始时记录左端点，在更新答案时保存当前左右端点。

2.8 固定模式串的子序列计数

统计文本 `s` 中模式串 `t` 作为子序列出现多少次。倒序更新防止同一文本字符被使用多次。

- `subseq_cnt(s,t,mod)`：结果对 `mod` 取模。
- 复杂度 `O(|s||t|)`，空间 `O(|t|)`。

```cpp
ll subseq_cnt(const string& s, const string& t, ll mod) { // s: 文本; t: 模式
    vector<ll> f(t.size() + 1);
    f[0] = 1;
    for (char c : s)
        for (int j = (int)t.size(); j >= 1; j--)
            if (c == t[j - 1]) f[j] = (f[j] + f[j - 1]) % mod;
    return f[t.size()];
}
```

2.9 不同非空子序列个数

`f[i]` 表示前 `i` 个字符的不同子序列数，包含空串。加入字符 `c` 时本应翻倍，但上次出现 `c` 时产生过的那部分会重复。

```cpp
ll distinct_subseq(const string& s, ll mod) { // 返回不同非空子序列数
    vector<ll> f(s.size() + 1);
    array<int, 256> last{};                    // 上次位置+1，0 表示未出现
    f[0] = 1;
    for (int i = 1; i <= (int)s.size(); i++) {
        unsigned char c = s[i - 1];
        f[i] = f[i - 1] * 2 % mod;
        if (last[c]) f[i] = (f[i] - f[last[c] - 1] + mod) % mod;
        last[c] = i;
    }
    return (f[s.size()] - 1 + mod) % mod;
}
```

---

3. 区间 DP

区间 DP 通常按长度从小到大枚举。状态 `f[l][r]` 只依赖更短区间，循环顺序一般为 `len -> l -> 断点 k`。

3.1 线性石子合并

相邻两堆才能合并，合并代价为两堆总重量，求把全部合成一堆的最小代价。

- `a` 为各堆重量，允许为 `ll`。
- 状态：`f[l][r]` 为合并闭区间 `[l,r]` 的最小代价。
- 转移：枚举最后一次把 `[l,k]` 与 `[k+1,r]` 合并。
- 复杂度 `O(n^3)`，空间 `O(n^2)`。

```cpp
ll merge_stone(const vector<ll>& a) { // 线性相邻合并最小代价
    int n = a.size();
    vector<ll> s(n + 1);
    for (int i = 0; i < n; i++) s[i + 1] = s[i] + a[i];
    vector<vector<ll>> f(n, vector<ll>(n));
    for (int len = 2; len <= n; len++)
        for (int l = 0, r = len - 1; r < n; l++, r++) {
            f[l][r] = INF;
            for (int k = l; k < r; k++)
                f[l][r] = min(f[l][r], f[l][k] + f[k + 1][r] + s[r + 1] - s[l]);
        }
    return n ? f[0][n - 1] : 0;
}
```

非负石子重量满足 Knuth 优化条件，可把时间降到 `O(n^2)`，见 9.3。

3.2 环形石子合并

把数组复制一遍，计算所有长度不超过 `n` 的区间，最后在 `n` 个起点中取最值。下面同时求最小和最大代价，复杂度 `O(n^3)`。

```
pair<ll, ll> merge_ring(vector<ll> a) { // 返回 {最小代价,最大代价}
    int n = a.size();
    a.insert(a.end(), a.begin(), a.end());
    vector<ll> s(2 * n + 1);
    for (int i = 0; i < 2 * n; i++) s[i + 1] = s[i] + a[i];
    vector f0(2 * n, vector<ll>(2 * n));
    vector f1(2 * n, vector<ll>(2 * n));
    for (int len = 2; len <= n; len++)
        for (int l = 0, r = len - 1; r < 2 * n; l++, r++) {
            f0[l][r] = INF, f1[l][r] = -INF;
            for (int k = l; k < r; k++) {
                ll w = s[r + 1] - s[l];
                f0[l][r] = min(f0[l][r], f0[l][k] + f0[k + 1][r] + w);
                f1[l][r] = max(f1[l][r], f1[l][k] + f1[k + 1][r] + w);
            }
        }
    ll mn = INF, mx = -INF;
    for (int l = 0; l < n; l++) mn = min(mn, f0[l][l + n - 1]), mx = max(mx, f1[l][l
+ n - 1]);
    return {mn, mx};
}
```

3.3 矩阵链乘

矩阵 `i` 的维度为 `p[i] × p[i+1]`。求改变括号顺序后的最少标量乘法次数。

- 参数 `p` 长度为矩阵数 `n+1`。
- `f[l][r]` 表示矩阵 `l..r` 的最少代价。
- 复杂度 `O(n^3)`，空间 `O(n^2)`。

```
ll matrix_chain(const vector<ll>& p) { // 第 i 个矩阵为 p[i] x p[i+1]
    int n = (int)p.size() - 1;
    vector<vector<ll>> f(n, vector<ll>(n));
    for (int len = 2; len <= n; len++)
        for (int l = 0, r = len - 1; r < n; l++, r++) {
            f[l][r] = INF;
            for (int k = l; k < r; k++)
                f[l][r] = min(f[l][r], f[l][k] + f[k + 1][r]
                              + p[l] * p[k + 1] * p[r + 1]);
        }
    return n ? f[0][n - 1] : 0;
}
```

若维度可到 `1e9`，三数乘积可能超过 `ll`，要用 `__int128` 比较并按题意处理溢出。

3.4 最长回文子序列

`f[l][r]` 表示区间内最长回文子序列长度。注意这是子序列，不要求连续。

```cpp
int lps(const string& s) { // 最长回文子序列长度
    int n = s.size();
    vector<vector<int>> f(n, vector<int>(n));
    for (int i = n - 1; i >= 0; i--) {
        f[i][i] = 1;
        for (int j = i + 1; j < n; j++)
            if (s[i] == s[j]) f[i][j] = f[i + 1][j - 1] + 2;
            else f[i][j] = max(f[i + 1][j], f[i][j - 1]);
    }
    return n ? f[0][n - 1] : 0;
}
```

当 `j=i+1` 且两端相等时访问 `f[i+1][i]`。由于整张表初始为 `0` 且下标仍在范围内，该写法成立。复杂度 `O(n^2)`。

3.5 戳气球型区间 DP

每次删除一个元素，收益取决于它当前两侧未删除元素。逆向考虑区间中**最后删除**谁，左右两边便相互独立。

下面是经典戳气球：气球值为非负数，戳 `k` 得到 `a[l]*a[k]*a[r]`，两端补 `1`，`f[l][r]` 表示开区间 `(l,r)` 的最大收益。

```cpp
ll burst(vector<ll> a) { // a: 气球值
    a.insert(a.begin(), 1); a.push_back(1);
    int n = a.size();
    vector<vector<ll>> f(n, vector<ll>(n));
    for (int len = 2; len < n; len++)
        for (int l = 0, r = l + len; r < n; l++, r++)
            for (int k = l + 1; k < r; k++)
                f[l][r] = max(f[l][r], f[l][k] + f[k][r] + a[l] * a[k] * a[r]);
    return f[0][n - 1];
}
```

---

4. 数位 DP

数位 DP 适合统计区间内满足"逐位可判断"限制的整数，例如不含某数字、数位和、相邻位关系、模数、禁止子串。通常先写 `calc(x)` 统计 `[0,x]`，区间 `[L,R]` 的答案为 `calc(R)-calc(L-1)`。

通用状态含义：

- `p`：当前处理到第几位。
- `st`：与题意有关的压缩状态。
- `lim`：前缀是否贴着上界；只有 `lim=0` 时可记忆化。

- `lead`：是否仍全是前导零；前导零通常不应触发普通数字转移。

4.1 自动机式通用数位 DP

把题目限制写成有限状态自动机。`go[st][d]` 表示在状态 `st` 填有效数字 `d` 后的新状态，`-1` 表示非法；`ok[st]` 表示结束时该状态是否计入。

- `DigitDP(go,ok)`：十进制自动机，初态固定为 `0`。
- `run(x)`：统计 `[1,x]` 中合法正整数；不含数字 `0` 本身。
- `S` 为自动机状态数。复杂度 `O(位数 × S × 10)`，空间 `O(位数 × S)`。

```
struct DigitDP {
    vector<array<int, 10>> go;
    vector<char> ok;
    vector<int> a;
    vector<vector<ll>> mem;

    DigitDP(vector<array<int, 10>> go, vector<char> ok)
        : go(move(go)), ok(move(ok)) {}

    ll dfs(int p, int st, bool lim, bool lead) {
        if (p == (int)a.size()) return !lead && ok[st];
        if (!lim && !lead && mem[p][st] != -1) return mem[p][st];
        int up = lim ? a[p] : 9;
        ll ans = 0;
        for (int d = 0; d <= up; d++) {
            bool nl = lead && d == 0;
            if (nl) ans += dfs(p + 1, st, lim && d == up, true);
            else if (go[st][d] != -1)
                ans += dfs(p + 1, go[st][d], lim && d == up, false);
        }
        if (!lim && !lead) mem[p][st] = ans;
        return ans;
    }

    ll run(ll x) { // x: 上界; 返回 [1,x] 合法数个数
        if (x <= 0) return 0;
        string s = to_string(x);
        a.clear();
        for (char c : s) a.push_back(c - '0');
        mem.assign(a.size(), vector<ll>(go.size(), -1));
        return dfs(0, 0, true, true);
    }
};
```

例如统计数位和模 `m` 等于 `r`：建立 `m` 个状态，令 `go[s][d]=(s+d)%m`，仅 `ok[r]=1`。统计 `[L,R]` 用 `dp.run(R)-dp.run(L-1)`。如果 `0` 也应算合法，需要根据题意额外加 `1`，因为模板刻意排除了全前导零。

若答案取模，在两处 `ans += ...` 后取模。若总数可能超过 `ll`，返回类型改为 `__int128` 或大整数。

4.2 Aho-Corasick + 数位 DP：禁止多个子串

先把十进制禁串插入 AC 自动机，失败指针建好后，每个节点和每个数字都有确定转移；将匹配到禁串的状态标为非法，就能直接交给上一模板。

```
struct AC10 {
    vector<array<int, 10>> tr{{}};
    vector<int> fail{0};
    vector<char> bad{0};

    void add(const string& s) { // s: 一个十进制禁串
        int u = 0;
        for (char c : s) {
            int d = c - '0';
            if (!tr[u][d]) {
                tr[u][d] = tr.size();
                tr.push_back({}); fail.push_back(0); bad.push_back(0);
            }
            u = tr[u][d];
        }
        bad[u] = 1;
    }

    void build() {
        queue<int> q;
        for (int d = 0; d < 10; d++) if (tr[0][d]) q.push(tr[0][d]);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            bad[u] |= bad[fail[u]];
            for (int d = 0; d < 10; d++) {
                if (tr[u][d]) fail[tr[u][d]] = tr[fail[u]][d], q.push(tr[u][d]);
                else tr[u][d] = tr[fail[u]][d];
            }
        }
    }

    DigitDP make_dp() { // build 后调用
        vector<array<int, 10>> go = tr;
        vector<char> ok(tr.size());
        for (int u = 0; u < (int)tr.size(); u++) {
            ok[u] = !bad[u];
            for (int d = 0; d < 10; d++) if (bad[go[u][d]]) go[u][d] = -1;
        }
        return DigitDP(move(go), move(ok));
    }
};
```

前导零不会进入自动机，因此禁串 `"0"` 不会自动排除整数 `0`；这和 `DigitDP` 只统计正整数的约定一致。

4.3 统计某数字出现总次数

`digit_cnt(x,d)` 返回 `[0,x]` 的十进制表示中数字 `d` 出现的总次数。前导零不计，但整数 `0` 的表示含一个 `0`。

DFS 同时返回 `{数字个数, d 的出现总数}`。某一位填 `d` 时，该分支中每个后缀对应的数都新增一次出现。

```
struct DigitCnt {
    int d;
    vector<int> a;
    pair<ll, ll> mem[20][2];
    bool vis[20][2];

    pair<ll, ll> dfs(int p, bool lim, bool lead) {
        if (p == (int)a.size()) return {1, 0};
        if (!lim && vis[p][lead]) return mem[p][lead];
        int up = lim ? a[p] : 9;
        ll cnt = 0, sum = 0;
        for (int x = 0; x <= up; x++) {
            bool nl = lead && x == 0;
            auto [c, s] = dfs(p + 1, lim && x == up, nl);
            cnt += c, sum += s + (x == d && !nl) * c;
        }
        if (!lim) vis[p][lead] = 1, mem[p][lead] = {cnt, sum};
        return {cnt, sum};
    }

    ll run(ll x, int dig) { // dig: 0..9; 返回 [0,x] 的出现次数
        if (x < 0) return 0;
        d = dig; a.clear();
        for (char c : to_string(x)) a.push_back(c - '0');
        memset(vis, 0, sizeof vis);
        ll ans = dfs(0, true, true).second;
        return ans + (d == 0);                  // 整数 0 自身
    }
};
```

区间 `[L,R]` 中的次数为 `run(R,d)-run(L-1,d)`。若 `R` 接近 `1e18-1`，总出现次数可能超过 `ll`，需改为 `__int128`。

---

5. 状态压缩 DP

当 `n` 约为 `20` 时，可用二进制集合表示已选元素。常用操作：

```cpp
bool has = s >> i & 1;                          // s 是否包含 i
int add = s | (1 << i);                         // 加入 i
int del = s ^ (1 << i);                         // 已知含 i 时删除 i
int cnt = __builtin_popcount((unsigned)s);      // 1 的个数
for (int t = s; ; t = (t - 1) & s) {            // 枚举 s 的所有子集，含 0 和 s
    // use(t)
    if (t == 0) break;
}
for (int t = s; t; t = (t - 1) & s) {           // 枚举非空子集
    // use(t)
}
```

所有集合及其所有子集的总枚举次数为 `3^n`，不是 `4^n`。

5.1 Hamilton 路径 / TSP

`f[s][u]` 表示从固定起点 `st` 出发，恰好访问集合 `s`，当前停在 `u` 的最小代价。适合 `n<=20` 左右。

- `w[u][v]`：完整代价矩阵；不存在的边填 `INF`。
- `tsp_path(w,st,ed)`：每点恰访问一次，从 `st` 到 `ed` 的最短 Hamilton 路径。
- 复杂度 `O(n^2 2^n)`，空间 `O(n2^n)`。

```cpp
ll tsp_path(const vector<vector<ll>>& w, int st, int ed) { // 点为 0..n-1
    int n = w.size(), N = 1 << n;
    vector<vector<ll>> f(N, vector<ll>(n, INF));
    f[1 << st][st] = 0;
    for (int s = 0; s < N; s++) {
        if (!(s >> st & 1)) continue;
        for (int u = 0; u < n; u++) if ((s >> u & 1) && f[s][u] != INF)
            for (int v = 0; v < n; v++) if (!(s >> v & 1) && w[u][v] != INF)
                f[s | (1 << v)][v] = min(f[s | (1 << v)][v], f[s][u] + w[u][v]);
    }
    return f[N - 1][ed];
}
```

求回到起点的 TSP 环时，答案取 `min(f[N-1][u]+w[u][st])`。可通过只保留包含 `st` 的状态、扁平数组和预枚举未选位降低常数。

5.2 最小权完美指派

`n` 个人与 `n` 个任务一一匹配，`w[i][j]` 是第 `i` 个人做任务 `j` 的代价。状态只需记录已使用任务集合，当前人编号为 `popcount(s)`。

- 复杂度 `O(n2^n)`，空间 `O(2^n)`；比二维 TSP 状态更省。

```cpp
ll assign_min(const vector<vector<ll>>& w) { // w[i][j]: 人 i 做任务 j
    int n = w.size(), N = 1 << n;
    vector<ll> f(N, INF); f[0] = 0;
    for (int s = 0; s < N; s++) {
        int i = __builtin_popcount((unsigned)s);
        if (i == n || f[s] == INF) continue;
        for (int j = 0; j < n; j++) if (!(s >> j & 1))
            f[s | (1 << j)] = min(f[s | (1 << j)], f[s] + w[i][j]);
    }
    return f[N - 1];
}
```

把 `min/INF` 改为方案数加法即可统计二分图完美匹配数：只有存在边 `(i,j)` 时才转移。

5.3 集合划分 DP

将全集划分成若干合法组，`cost[t]` 是一组恰为子集 `t` 的代价，求最小总代价。直接枚举子集为 `O(3^n)`。

固定枚举 `s` 的最低位必须在 `t` 中，可避免同一划分因组的顺序不同而重复，计数题尤其重要。

```cpp
vector<ll> f(1 << n, INF); f[0] = 0;
for (int s = 1; s < (1 << n); s++) {
    int b = s & -s;                                 // 固定最低位所在组
    for (int t = s; t; t = (t - 1) & s)
        if ((t & b) && cost[t] != INF)
            f[s] = min(f[s], f[s ^ t] + cost[t]);
}
// f[(1<<n)-1]: 最小划分代价
```

5.4 行状态 DP：互不相邻选格

适合网格宽度较小、行数较大，限制只涉及同一行及相邻行。先筛出行内合法状态，再预处理状态间兼容关系。

下面求非空带权网格中选格最大权值，要求上下左右均不能同时选；`a[i][j]` 为权值，可为负。

- `n` 行、`m` 列，适合 `m<=15` 左右。
- 复杂度 `O(nS^2+Sm)`，`S` 是无相邻 1 的合法状态数。

```cpp
ll grid_independent(const vector<vector<ll>>& a) { // 不选也合法
    int n = a.size(), m = a[0].size();
    vector<int> st;
    for (int s = 0; s < (1 << m); s++) if (!(s & (s << 1))) st.push_back(s);
    int S = st.size();
    vector<vector<int>> pre(S);
    for (int i = 0; i < S; i++)
        for (int j = 0; j < S; j++) if (!(st[i] & st[j])) pre[i].push_back(j);
    vector<ll> f(S, -INF), g(S, -INF); f[0] = 0;
    for (int r = 0; r < n; r++) {
```

```
        fill(g.begin(), g.end(), -INF);
        for (int i = 0; i < S; i++) {
            ll w = 0;
            for (int c = 0; c < m; c++) if (st[i] >> c & 1) w += a[r][c];
            for (int j : pre[i]) if (f[j] != -INF) g[i] = max(g[i], f[j] + w);
        }
        f.swap(g);
    }
    return *max_element(f.begin(), f.end());
}
```

若还禁止斜对角相邻，兼容条件再加 `!(st[i]&(st[j]<<1))` 和 `!(st[i]&(st[j]>>1))`。

5.5 轮廓线 DP：多米诺骨牌铺满

统计用 `1×2` 骨牌铺满 `n×m` 棋盘的方案数。逐格扫描，状态 `s` 的最低位表示当前格是否已被前面伸出的骨牌占用。

- 为降低状态数，让 `m=min(n,m)`。
- `n,m` 均为正数；无障碍棋盘；答案不取模，可能溢出 `ll`。
- 复杂度 `O(nm2^m)`，空间 `O(2^m)`。

```
ll domino(int n, int m) { // n*m 无障碍棋盘
    if (n < m) swap(n, m);
    int N = 1 << m;
    vector<ll> f(N), g(N); f[0] = 1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            fill(g.begin(), g.end(), 0);
            for (int s = 0; s < N; s++) if (f[s]) {
                if (s & 1) g[s >> 1] += f[s];              // 当前格已占
                else {
                    g[(s >> 1) | (1 << (m - 1))] += f[s]; // 竖放，伸到下一行
                    if (j + 1 < m && !(s & 2)) g[(s >> 1) | 1] += f[s]; // 横放
                }
            }
            f.swap(g);
        }
    return f[0];
}
```

加入障碍时，应在处理格子前把障碍占用并禁止骨牌伸入障碍；不同题目的状态时序很容易差一位，建议先用 `1×1`、`1×2`、`2×2` 手算验证。

5.6 Steiner Tree：少量终端最小连通子图

给定非负权无向图和 `k` 个终端，求连接全部终端的最小权连通子图。状态 `f[s][v]` 表示已经连接终端集合 `s`，且当前连通结构在点 `v` 汇合的最小代价。

每个状态先枚举子集在同一点合并，再以全部 `f[s][v]` 为多源起点跑 Dijkstra。`steiner(n,e,t)` 中边为 `{u,v,w}`，终端数组 `t` 存点编号；返回 `INF` 表示无法全部连通。

复杂度 $O(3^k n + 2^k (m+n)\log n)$，空间 $O(2^k n)$，通常适合 `k<=12` 左右。若原图是树，直接取终端虚树或删去无终端叶子更快，不应使用本段。

```
ll steiner(int n, const vector<tuple<int, int, ll>> &e,
           const vector<int> &t) {
    // n：点数；e：非负权无向边；t：必须连通的终端
    int k = t.size();
    if (!k) return 0;
    int N = 1 << k;
    vector<vector<pair<int, ll>>> g(n + 1);
    for (auto [u, v, w] : e)
        g[u].push_back({v, w}), g[v].push_back({u, w});
    vector<vector<ll>> f(N, vector<ll>(n + 1, INF));
    for (int i = 0; i < k; i++) f[1 << i][t[i]] = 0;

    for (int s = 1; s < N; s++) {
        for (int a = (s - 1) & s; a; a = (a - 1) & s) {
            int b = s ^ a;
            if (!b || a > b) continue;
            for (int v = 1; v <= n; v++)
                if (f[a][v] < INF && f[b][v] < INF && f[b][v] <= INF - f[a][v])
                    f[s][v] = min(f[s][v], f[a][v] + f[b][v]);
        }
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>>
q;
        for (int v = 1; v <= n; v++) if (f[s][v] < INF) q.push({f[s][v], v});
        while (!q.empty()) {
            auto [du, u] = q.top(); q.pop();
            if (du != f[s][u]) continue;
            for (auto [v, w] : g[u]) if (du <= INF - w && f[s][v] > du + w)
                f[s][v] = du + w, q.push({f[s][v], v});
        }
    }
    return *min_element(f[N - 1].begin() + 1, f[N - 1].end());
}
```

---

6. SOS DP 与子集变换

设全集有 `n` 个元素，数组长度 `N=2^n`。SOS DP 能在 `O(n2^n)` 内计算每个集合的所有子集或超集聚合，远快于逐集合枚举子集的 `O(3^n)`。

6.1 子集和：子集 Zeta 变换

输入 `a[s]`，输出

$f[s] = \sum_{t \subseteq s} a[t].$

- `subset_zeta(f,n)` 原地变换，`f.size()` 必须为 `2^n`。
- 加法可换成其他满足结合律的聚合，如 `max/min`；但只有可逆运算才有对应逆变换。
- 复杂度 `O(n2^n)`，额外空间 `O(1)`。

```
template<class T>
void subset_zeta(vector<T>& f, int n) { // f[s] <- sum_{t subset s} f[t]
    for (int i = 0; i < n; i++)
        for (int s = 0; s < (1 << n); s++)
            if (s >> i & 1) f[s] += f[s ^ (1 << i)];
}
```

6.2 子集 Möbius 逆变换

已知 `F[s]=sum_{t subseteq s} a[t]`，恢复原数组 `a`。顺序与 Zeta 相同，把加法改成减法即可。

```
template<class T>
void subset_mobius(vector<T>& f, int n) { // 子集和的逆变换
    for (int i = 0; i < n; i++)
        for (int s = 0; s < (1 << n); s++)
            if (s >> i & 1) f[s] -= f[s ^ (1 << i)];
}
```

取模时每次减法后规范到 `[0,mod)`；也可以最后统一规范，但中间值必须保证不溢出。

6.3 超集 Zeta 与 Möbius

超集和为

$f[s] = \sum_{t \supseteq s} a[t].$

只需把转移条件从“当前位为 1”改成“当前位为 0”。

```cpp
template<class T>
void superset_zeta(vector<T>& f, int n) { // f[s] <- sum_{t superset s} f[t]
    for (int i = 0; i < n; i++)
        for (int s = 0; s < (1 << n); s++)
            if (!(s >> i & 1)) f[s] += f[s ^ (1 << i)];
}

template<class T>
void superset_mobius(vector<T>& f, int n) { // 超集和的逆变换
    for (int i = 0; i < n; i++)
        for (int s = 0; s < (1 << n); s++)
            if (!(s >> i & 1)) f[s] -= f[s ^ (1 << i)];
}
```

6.4 每个集合的最优子集

若 `val[t]` 是子集 `t` 的得分，要求每个 `s` 的最高分子集及其编号，把聚合改为比较即可。

```cpp
vector<pair<ll, int>> f(1 << n);
for (int s = 0; s < (1 << n); s++) f[s] = {val[s], s};
for (int i = 0; i < n; i++)
    for (int s = 0; s < (1 << n); s++) if (s >> i & 1)
        f[s] = max(f[s], f[s ^ (1 << i)]);     // pair 同分时取较大编号
// f[s] = {最高分, 对应子集}
```

若同分要取较小编号，自定义比较，不能依赖 `pair` 默认顺序。

6.5 子集卷积

给定 `a,b`，计算

$c[s] = \sum_{t \subseteq s} a[t] b[s \setminus t].$

按集合大小分层做 Zeta 变换，在每个状态上卷积大小维，再 Möbius 逆变换。适合 `n<=20` 左右。

- 输入长度均为 `2^n`，返回结果对 `mod` 取模。
- 复杂度 `O(n^2 2^n)`，空间 `O(n2^n)`。

```cpp
vector<ll> subset_conv(const vector<ll>& a, const vector<ll>& b, int n, ll mod) {
    int N = 1 << n;
    vector<vector<ll>> A(n + 1, vector<ll>(N)), B = A, C = A;
    for (int s = 0; s < N; s++) {
        int k = __builtin_popcount((unsigned)s);
        A[k][s] = (a[s] % mod + mod) % mod;
        B[k][s] = (b[s] % mod + mod) % mod;
    }
    for (int k = 0; k <= n; k++) {
        for (int i = 0; i < n; i++)
            for (int s = 0; s < N; s++) if (s >> i & 1) {
                A[k][s] = (A[k][s] + A[k][s ^ (1 << i)]) % mod;
```

```
                B[k][s] = (B[k][s] + B[k][s ^ (1 << i)]) % mod;
            }
    }
    for (int s = 0; s < N; s++)
        for (int k = 0; k <= n; k++)
            for (int i = 0; i <= k; i++)
                C[k][s] = (C[k][s] + A[i][s] * B[k - i][s]) % mod;
    for (int k = 0; k <= n; k++)
        for (int i = 0; i < n; i++)
            for (int s = 0; s < N; s++) if (s >> i & 1)
                C[k][s] = (C[k][s] - C[k][s ^ (1 << i)] + mod) % mod;
    vector<ll> c(N);
    for (int s = 0; s < N; s++) c[s] = C[__builtin_popcount((unsigned)s)][s];
    return c;
}
```

乘法 `A*B` 若 `mod` 接近 `1e18` 会溢出 `ll`，需用 `__int128`。

---

7. 树形 DP

以下模板默认树的点编号为 `1..n`。递归 DFS 在链状大树上可能爆栈；`n` 达到数十万时可改用显式栈求父亲和遍历序，再逆序 DP。

7.1 树的直径：边权可为负

`d[u]` 表示从 `u` 向子树走、至少包含 `u` 的最大单臂收益。允许一条边都不选，因此答案至少为 `0`；若题目要求路径必须包含边且全为负，需要单独调整初值。

- `g[u]` 存 `{v,w}`，无向边加两次。
- 复杂度 `O(n)`。

```
vector<vector<pair<int, ll>>> g;
vector<ll> d;
ll dia;

void dfs_dia(int u, int p) { // u: 当前点; p: 父亲
    d[u] = 0;
    for (auto [v, w] : g[u]) if (v != p) {
        dfs_dia(v, u);
        ll x = d[v] + w;
        dia = max(dia, d[u] + x);
        d[u] = max(d[u], x);
    }
}
// d.assign(n+1,0), dia=0, dfs_dia(1,0)
```

无负权时也可用两次 DFS/BFS，但上面的 DP 更容易扩展到点权、计数等变体。

7.2 树上最大权独立集

相邻点不能同时选。`f[u][0/1]` 表示不选/选择 `u` 时，`u` 子树的最大权值。

- `w[u]` 为点权，可以为负；空集合法。
- 复杂度 `O(n)`，空间 `O(n)`。

```
vector<vector<int>> g;
vector<ll> w;
vector<array<ll, 2>> f;

void dfs_mis(int u, int p) { // 根任选
    f[u] = {0, w[u]};
    for (int v : g[u]) if (v != p) {
        dfs_mis(v, u);
        f[u][0] += max(f[v][0], f[v][1]);
        f[u][1] += f[v][0];
    }
}
// 答案 max(f[root][0],f[root][1])
```

7.3 树上背包：选恰好 `k` 个点

`f[u][j]` 表示在 `u` 子树恰选 `j` 个点的最大权值。逐个合并孩子，相当于对子树做分组背包。

- 下面允许任意选点，没有父子依赖；`w[u]` 为点权。
- `sz[u]` 限制有效枚举范围，能显著减少常数。
- 最坏复杂度 `O(nk^2)`，实际合并复杂度与树形有关；空间 `O(nk)`。

```
int K;
vector<vector<int>> g;
vector<ll> w;
vector<vector<ll>> f;
vector<int> sz;

void dfs_tk(int u, int p) { // f[u][j]: 子树 u 恰选 j 点
    f[u].assign(K + 1, -INF);
    f[u][0] = 0;
    if (K >= 1) f[u][1] = w[u];
    sz[u] = 1;
    for (int v : g[u]) if (v != p) {
        dfs_tk(v, u);
        vector<ll> h(K + 1, -INF);
        for (int i = 0; i <= min(K, sz[u]); i++) if (f[u][i] != -INF)
            for (int j = 0; j <= min(K - i, sz[v]); j++) if (f[v][j] != -INF)
                h[i + j] = max(h[i + j], f[u][i] + f[v][j]);
        sz[u] += sz[v];
        f[u].swap(h);
    }
```

```
}
// dfs_tk(root,0); f[root][K] 为答案
```

如果"选择子节点必须选择父节点"，合并时保留 `f[u][0]=0`，但只有 `i>0` 时才能从孩子选择 `j>0`；或者把 `u` 预先选入后仅维护已选 `u` 的状态。

7.4 换根 DP：每个点到所有点距离和

先以 `1` 为根求子树大小 `sz[u]` 和根的距离和 `ans[1]`。从父亲 `u` 换根到孩子 `v` 时，`v` 子树内 `sz[v]` 个点距离减 `1`，其余点距离加 `1`：

$$ans[v] = ans[u] + n - 2sz[v].$$

- 无权树，返回每个点到所有点的距离和。
- 复杂度 `O(n)`。

```
vector<ll> sum_dist(const vector<vector<int>>& g) { // g: 1-index 无权树
    int n = (int)g.size() - 1;
    vector<int> p(n + 1), sz(n + 1, 1), ord{1};
    for (int i = 0; i < (int)ord.size(); i++) {
        int u = ord[i];
        for (int v : g[u]) if (v != p[u]) p[v] = u, ord.push_back(v);
    }
    vector<ll> ans(n + 1);
    vector<int> dep(n + 1);
    for (int u : ord) {
        if (u != 1) dep[u] = dep[p[u]] + 1;
        ans[1] += dep[u];
    }
    for (int i = n - 1; i > 0; i--) sz[p[ord[i]]] += sz[ord[i]];
    for (int u : ord) if (u != 1) ans[u] = ans[p[u]] + n - 2LL * sz[u];
    return ans;
}
```

7.5 通用换根：树上最远距离

求每个点到其他点的最远距离。第一遍求向下最长 `dn[u]` 和次长值；第二遍把父方向的最优值 `up[u]` 传给孩子。边有非负权。

- `g[u]` 存 `{v,w}`。
- 复杂度 `O(n)`。

```
vector<ll> farthest(const vector<vector<pair<int, ll>>>& g) { // 1-index 非负权树
    int n = (int)g.size() - 1;
    vector<int> p(n + 1), ord{1}, who(n + 1);
    vector<ll> pw(n + 1), dn(n + 1), dn2(n + 1), up(n + 1);
    for (int i = 0; i < (int)ord.size(); i++) {
        int u = ord[i];
        for (auto [v, w] : g[u]) if (v != p[u])
            p[v] = u, pw[v] = w, ord.push_back(v);
```

```
    }
    for (int i = n - 1; i > 0; i--) {
        int v = ord[i], u = p[v];
        ll x = dn[v] + pw[v];
        if (x > dn[u]) dn2[u] = dn[u], dn[u] = x, who[u] = v;
        else if (x > dn2[u]) dn2[u] = x;
    }
    for (int u : ord) if (u != 1) {
        ll best = (who[p[u]] == u ? dn2[p[u]] : dn[p[u]]);
        up[u] = max(up[p[u]], best) + pw[u];
    }
    vector<ll> ans(n + 1);
    for (int u = 1; u <= n; u++) ans[u] = max(dn[u], up[u]);
    return ans;
}
```

8. DAG DP

只有有向无环图才能直接按拓扑序 DP。若原图可能有环，应先缩点成 DAG，或换用其他建模。

8.1 DAG 最长路与方案数

从源点 `s` 到所有点，求最大路径权和及达到最大值的路径数。`g[u]` 存 `{v,w}`。

- 不可达距离为 `-INF`，计数为 `0`。
- 平行边按不同边产生不同路径；若题目按点序列去重，要先合并平行边。
- 复杂度 `O(n+m)`。

```
pair<vector<ll>, vector<ll>> dag_longest(
    const vector<vector<pair<int, ll>>>& g, int s, ll mod) { // g: 1-index
    int n = (int)g.size() - 1;
    vector<int> in(n + 1);
    for (int u = 1; u <= n; u++) for (auto [v, w] : g[u]) in[v]++;
    queue<int> q;
    for (int u = 1; u <= n; u++) if (!in[u]) q.push(u);
    vector<ll> d(n + 1, -INF), c(n + 1); d[s] = 0, c[s] = 1;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto [v, w] : g[u]) {
            if (d[u] != -INF) {
                ll x = d[u] + w;
                if (x > d[v]) d[v] = x, c[v] = c[u];
                else if (x == d[v]) c[v] = (c[v] + c[u]) % mod;
            }
            if (!--in[v]) q.push(v);
        }
    }
    return {d, c};
}
```

若处理完的点数少于 `n`，图中有环，模板结果不成立。求最短路把 `max/-INF` 改为 `min/INF` 即可；DAG 中允许负边。

8.2 DAG 路径总数

统计从 `s` 到 `t` 的路径数。拓扑序中令 `f[s]=1`，对每条边执行 `f[v]+=f[u]`。如果要统计从每个点到 `t`，按逆拓扑序令 `f[t]=1`，执行 `f[u]+=f[v]`。

```
vector<ll> f(n + 1); f[s] = 1;
for (int u : topo)                              // topo: 正拓扑序
    for (int v : g[u])
        f[v] = (f[v] + f[u]) % MOD;
// f[t]: s 到 t 的路径数
```

---

9. DP 优化

优化前先写出朴素转移并确认决策范围。分治、Knuth、单调队列和斜率优化都有前提；若单调性或代价结构不满足，模板会静默算错，而不是超时。

9.1 滚动数组与缓存友好

若第 `i` 层只依赖第 `i-1` 层，用两个一维数组：

```
vector<ll> f(m + 1), g(m + 1);
for (int i = 1; i <= n; i++) {
    fill(g.begin(), g.end(), INF);              // 计算第 i 层
    for (int j = 0; j <= m; j++) {
        // g[...] <- f[j]
    }
    f.swap(g);
}
```

原地压缩只有在依赖方向明确时才做。二维数组循环尽量让连续内存的下标放内层；C++ 行优先存储，`f[i][j]` 应优先让 `j` 连续变化。

9.2 单调队列优化滑动窗口转移

形如

$$g[i] = a[i] + \max_{i-k \le j < i} f[j]$$

时，维护 `f[j]` 单调递减的下标队列。每个下标入队、出队各一次。

- `win_dp(f,a,k)`：返回新一层；窗口不含 `i` 本身。
- 时间 `O(n)`，空间 `O(n)`。

```cpp
vector<ll> win_dp(const vector<ll>& f, const vector<ll>& a, int k) { // k: 最大回看距离
    int n = f.size();
    vector<ll> g(n, -INF);
    deque<int> q;
    for (int i = 0; i < n; i++) {
        while (!q.empty() && q.front() < i - k) q.pop_front();
        if (!q.empty() && f[q.front()] != -INF) g[i] = a[i] + f[q.front()];
        while (!q.empty() && f[q.back()] <= f[i]) q.pop_back();
        q.push_back(i);
    }
    return g;
}
```

最小值把比较方向反过来。若转移窗口包含 `i`，应先将 `i` 入队再查询；顺序由方程决定。

9.3 分治优化

适用于分层转移

$$dp[k][i] = \min_{j<i}\{dp[k-1][j] + w(j+1, i)\},$$

且最优决策 `opt[k][i]` 随 `i` 单调不减。常见于把序列分成恰好 `K` 个非空连续段。

- `dc_partition(n,K,cost)`：`cost(l,r)` 为闭区间 `[l,r]` 一组的代价，要求 `O(1)` 求值。
- 返回恰分 `K` 组覆盖 `1..n` 的最小代价。
- 时间通常 `O(Kn log n)`，空间 `O(n)`；某些更强条件下总枚举可达 `O(Kn)`。

```cpp
ll dc_partition(int n, int K, const function<ll(int, int)>& cost) {
    vector<ll> pre(n + 1, INF), cur(n + 1, INF);
    pre[0] = 0;
    for (int k = 1; k <= K; k++) {
        fill(cur.begin(), cur.end(), INF);
        auto solve = [&](auto&& self, int l, int r, int ql, int qr) -> void {
            if (l > r) return;
            int mid = (l + r) >> 1, best = -1;
            int R = min(qr, mid - 1);
            for (int j = ql; j <= R; j++) if (pre[j] != INF) {
                ll x = pre[j] + cost(j + 1, mid);
                if (x < cur[mid]) cur[mid] = x, best = j;
            }
            if (best == -1) best = ql;
            self(self, l, mid - 1, ql, best);
            self(self, mid + 1, r, best, qr);
        };
        if (k <= n) solve(solve, k, n, k - 1, n - 1);
        pre.swap(cur);
    }
    return pre[n];
}
```

最优决策单调性通常由四边形不等式或 Monge 性质推出。没有证明时至少用小规模随机数据对拍朴素 `O(Kn^2)`。

9.4 Knuth 优化

适用于区间转移

$$f[l][r]=\min_{l\le k<r}\{f[l][k]+f[k+1][r]\}+w(l,r),$$

且满足 `opt[l][r-1] <= opt[l][r] <= opt[l+1][r]`。非负权线性石子合并是经典应用。

- 下面直接求石子合并最小代价。
- 时间 `O(n^2)`，空间 `O(n^2)`。

```
ll merge_knuth(const vector<ll>& a) { // a[i]>=0
    int n = a.size();
    vector<ll> s(n + 1);
    for (int i = 0; i < n; i++) s[i + 1] = s[i] + a[i];
    vector f(n, vector<ll>(n));
    vector opt(n, vector<int>(n));
    for (int i = 0; i < n; i++) opt[i][i] = i;
    for (int len = 2; len <= n; len++)
        for (int l = 0, r = l + len - 1; r < n; l++, r++) {
            f[l][r] = INF;
            int L = opt[l][r - 1], R = min(r - 1, opt[l + 1][r]);
            for (int k = L; k <= R; k++) {
                ll x = f[l][k] + f[k + 1][r] + s[r + 1] - s[l];
                if (x < f[l][r]) f[l][r] = x, opt[l][r] = k;
            }
        }
    return n ? f[0][n - 1] : 0;
}
```

Knuth 的状态形式与分治优化不同，不能只因“决策看起来单调”就套用。

9.5 单调斜率优化

当转移能写成“查询若干直线在 `x` 处的最小值”，并且：

- 加入直线的斜率单调不增；
- 查询横坐标 `x` 单调不减；

可以用双端队列做到均摊 `O(1)` 加线和查询。直线为 `y=m*x+b`。

```
struct HullMin {
    struct L { ll m, b; ll get(ll x) const { return m * x + b; } };
    deque<L> q;

    static bool bad(const L& a, const L& b, const L& c) {
        return ((__int128)b.b - a.b) * ((__int128)b.m - c.m)
             >= ((__int128)c.b - b.b) * ((__int128)a.m - b.m);
```

```
    }
    void add(ll m, ll b) { // m 必须按不增顺序加入
        L z{m, b};
        if (!q.empty() && q.back().m == m) {
            if (q.back().b <= b) return;
            q.pop_back();
        }
        while (q.size() >= 2 && bad(q[q.size() - 2], q.back(), z)) q.pop_back();
        q.push_back(z);
    }
    ll qry(ll x) { // x 必须按不减顺序查询
        while (q.size() >= 2 && q[0].get(x) >= q[1].get(x)) q.pop_front();
        return q[0].get(x);
    }
};
```

交叉乘使用 `__int128` 防止判坏溢出，但 `m*x+b` 仍要求答案在 `ll` 内；否则 `get` 也改为 `__int128`。求最大值可把直线值整体取负，或统一反转比较。斜率、查询顺序任一不单调时用 Li Chao 树。

9.6 Li Chao 线段树

支持任意顺序加直线，在预先给定的离散查询横坐标上查询最小值。每次加线和查询 `O(log Q)`。

- `LiChao xs`：`xs` 包含之后所有可能查询的横坐标，构造时会排序去重。
- `add(m,b)`：加入 `y=mx+b`。
- `qry(x)`：查询最小值，`x` 必须在构造时的 `xs` 中。

```
struct LiChao {
    struct L {
        ll m = 0, b = INF;
        ll get(ll x) const { return b == INF ? INF : m * x + b; }
    };
    vector<ll> x;
    vector<L> tr;

    LiChao(vector<ll> x) : x(move(x)) {
        sort(this->x.begin(), this->x.end());
        this->x.erase(unique(this->x.begin(), this->x.end()), this->x.end());
        tr.resize(this->x.size() * 4 + 4);
    }
    void add(L z, int p, int l, int r) {
        int m = (l + r) >> 1;
        bool lef = z.get(x[l]) < tr[p].get(x[l]);
        bool mid = z.get(x[m]) < tr[p].get(x[m]);
        if (mid) swap(z, tr[p]);
        if (l == r) return;
        if (lef != mid) add(z, p << 1, l, m);
        else add(z, p << 1 | 1, m + 1, r);
    }
```

```
    void add(ll m, ll b) { // 加直线 y=m*x+b
        add({m, b}, 1, 0, x.size() - 1);
    }
    ll qry(int i, int p, int l, int r) const {
        ll ans = tr[p].get(x[i]);
        if (l == r) return ans;
        int m = (l + r) >> 1;
        if (i <= m) return min(ans, qry(i, p << 1, l, m));
        return min(ans, qry(i, p << 1 | 1, m + 1, r));
    }
    ll qry(ll v) const { // v 必须在 x 中
        int i = lower_bound(x.begin(), x.end(), v) - x.begin();
        return qry(i, 1, 0, x.size() - 1);
    }
};
```

构造时 `xs` 不能为空。若乘加可能溢出，需要把 `get` 和查询返回值改成 `__int128`。若需要在线出现未知横坐标，可改用动态开点、固定整数值域的 Li Chao 树。

9.7 WQS 二分（Aliens Trick）

当 DP 要求“恰好选 `K` 个”，但去掉数量限制后容易求解，可给每次选择附加代价 `x`，二分 `x`。DP 值维护二元组 `{原代价+x*次数, 次数}`，相同调整后代价时按需要偏向更多或更少次数，使选择次数随 `x` 单调。

最小化问题常用框架：

```
pair<ll, int> check(ll x) {
    // 返回 {加上每次 x 后的最小总代价, 对应选择次数}
}

ll l = L, r = R;                                    // 覆盖最优惩罚的整数范围
while (l < r) {
    ll m = l + (r - l) / 2;
    if (check(m).second <= K) r = m;
    else l = m + 1;
}
ll ans = check(l).first - l * K;
```

此框架要求问题具有离散凸性，并且 `check` 中同值时的次数偏好与二分方向一致。答案是否应在相邻两个惩罚处取值、能否恢复恰好 `K` 次，取决于题目；使用前必须证明。

---

10. 概率与期望 DP

10.1 DAG 上到终点的期望代价

从 `u` 等概率走一条出边，走边 `(u,v)` 付出 `w`，求到终点 `t` 的期望总代价。若所有边都沿拓扑序前进：

$$E[u] = \frac{1}{deg(u)} \sum_{(u,v)} (w + E[v]).$$

按逆拓扑序计算。若存在到不了 `t` 的路径，期望可能无穷，必须先做可达性判断。

```
vector<long double> expect_dag(
    const vector<vector<pair<int, ll>>>& g, const vector<int>& topo, int t) {
    int n = (int)g.size() - 1;
    vector<long double> f(n + 1);
    for (int i = (int)topo.size() - 1; i >= 0; i--) {
        int u = topo[i];
        if (u == t) continue;
        for (auto [v, w] : g[u]) f[u] += w + f[v];
        if (!g[u].empty()) f[u] /= g[u].size();
    }
    return f;
}
```

有环的随机过程一般会形成线性方程组，应使用高斯消元或利用特殊结构，不再是简单的拓扑 DP。

10.2 收集全部物品的期望步数

每次独立均匀抽取 `n` 种之一，已经收集 `i` 种时，抽到新品概率为 `(n-i)/n`。由

$$E_i = 1 + \frac{i}{n}E_i + \frac{n-i}{n}E_{i+1}$$

得到 `E_i=n/(n-i)+E_{i+1}`，即答案 `n(1+1/2+...+1/n)`。

```
long double coupon(int n) { // 均匀收集 n 种的期望抽取次数
    long double ans = 0;
    for (int k = 1; k <= n; k++) ans += (long double)n / k;
    return ans;
}
```

期望 DP 中若转移能回到自身，不能直接把 `E[u]` 当作已知量使用；应先把自环项移到方程左边。

---

11. 常见补充模型

11.1 最长合法括号子串

`f[i]` 表示以 `s[i]` 结尾的最长合法括号子串长度。仅适用于圆括号；多种括号时把配对判断抽成函数。

```cpp
pair<int, int> longest_parentheses(const string& s) { // 返回 {左端点,长度}
    int n = s.size(), best = 0, pos = 0;
    vector<int> f(n);
    for (int i = 1; i < n; i++) if (s[i] == ')') {
        int j = i - 1 - f[i - 1];
        if (j >= 0 && s[j] == '(') {
            f[i] = f[i - 1] + 2;
            if (j > 0) f[i] += f[j - 1];
            if (f[i] > best) best = f[i], pos = i - best + 1;
        }
    }
    return {pos, best};
}
```

只在严格变长时更新，因此同长度保留最靠左答案。

11.2 最大 `m` 段子段和

从数组中选恰好 `m` 个互不相交、非空连续子段，使总和最大。`f[j]` 表示上一层在前 `j` 个数中选完若干段的最优值；`end` 表示当前层最后一段必须以 `j` 结尾。

- 复杂度 `O(mn)`，空间 `O(n)`。

```cpp
ll max_m_segments(const vector<ll>& a, int m) { // 恰选 m 个非空段
    int n = a.size();
    vector<ll> pre(n + 1), cur(n + 1, -INF);
    for (int k = 1; k <= m; k++) {
        fill(cur.begin(), cur.end(), -INF);
        ll ed = -INF;
        for (int j = k; j <= n; j++) {
            ed = max(ed + a[j - 1], pre[j - 1] + a[j - 1]);
            cur[j] = (j == k ? ed : max(cur[j - 1], ed));
        }
        pre.swap(cur);
    }
    return pre[n];
}
```

若 `m>n` 则无解，返回 `-INF`。`ed+a[j-1]` 在 `ed=-INF` 时仍处在安全范围，因为本板子的 `INF=2^62` 留有余量。

11.3 子集内边权预处理

状压题常需快速得到集合内部总边权。令 `lb` 为 `s` 最低位对应点，利用 `cost[s]=cost[s去掉lb]+lb与其余点的边权和`，可从 `O(n^2 2^n)` 降为 `O(n2^n)`。

```cpp
vector<ll> cost(1 << n);
for (int s = 1; s < (1 << n); s++) {
    int i = __builtin_ctz((unsigned)s), t = s ^ (1 << i);
    cost[s] = cost[t];
    for (int j = 0; j < n; j++) if (t >> j & 1) cost[s] += w[i][j];
}
```

无向边只加一次；若 `w` 是有向贡献，按题意加入一个或两个方向。

---

12. 赛场使用与排错

12.1 复杂度粗估

- `O(nW)`：`nW` 在几千万量级通常可行；使用 `vector<vector<ll>>` 会增加访存常数。
- `O(n^2)`：`n` 通常几千；状态很轻时可更大。
- `O(n^3)`：`n` 通常几百以内。
- `O(n2^n)`：`n` 通常约 `20`；二维 `ll` 状态的内存也可能先超限。
- `O(3^n)`：`n` 通常约 `16~18`。
- 上述只是经验值，需结合时限、语言、状态常数和内存判断。

12.2 高频错误

1. 把“容量不超过”模板用于“恰好达到”，导致不可达状态被默认 `0`。
2. 01 背包容量正序，意外把一件物品选多次。
3. 分组背包先枚举物品再枚举容量，同组选择了多件。
4. 最大值 DP 从 `-INF` 转移时未判可达；最小值 DP 对 `INF` 继续加代价。
5. 状压使用 `1<<n` 时 `n>=31` 发生有符号整数溢出；大集合用 `1ULL<<n`。
6. `O(n2^n)` 状态开成 `vector<vector<ll>>` 导致大量小对象；极限内存下用一维连续数组 `f[s*n+u]`。
7. 数位 DP 缓存了 `limit=1` 的状态，或没区分前导零。
8. 区间 DP 未按区间长度递增，来源状态尚未计算。
9. 树形 DP 在链上递归爆栈；改用父亲数组和正/逆遍历序。
10. 未证明决策单调就套分治或 Knuth 优化。

12.3 对拍建议

为优化模板准备小规模暴力：背包枚举全部子集，状压划分递归枚举，区间 DP 枚举所有断点，优化 DP 与朴素转移比较。随机数据除比较答案外，还要覆盖空集、全负值、重复值、容量 `0`、单点树、不可达状态和取模边界。